

CS6530 – Applied Cryptography

Jul–Nov 2026

Assignment 2

Authenticated Ephemeral Key Establishment

X25519 · Ed25519 · SHA-256 · HKDF-SHA-256 · AES-256-GCM

REPORT BY

Name: Santhosh D R

Roll Number: CS26E004

Partner: Nipun Bargal (MD24B033)

Instructor: Dr. Manikantan Srinivasan

Department: Computer Science and Engineering

Date: September 2026

1. Design Summary

The goal of this assignment is to implement an authenticated ephemeral key establishment protocol between Alice and Bob over an untrusted TCP network. The protocol provides mutual authentication, forward secrecy, replay resistance, and MITM resistance using the fixed cryptographic suite: X25519, Ed25519, SHA-256, HKDF-SHA-256, and AES-256-GCM.

1.1 System Model and Assumptions

- Alice runs on one machine; Bob runs on another. The TCP helper (*transport.py*) provides framing only — no confidentiality, authentication, or integrity.
- Long-term Ed25519 keypairs are generated once and pre-shared out-of-band. No PKI or certificate chain is required.
- X25519 and Ed25519 are kept on separate key pairs and are never reused or converted between roles.
- Mallory (for TR-2) runs as a third process intercepting the TCP connection. A third physical machine is not required.

1.2 Four-Message Handshake Design (M1–M4)

M1 (Alice → Bob):

- Alice generates a fresh 16-byte *Alice_SID* and an ephemeral X25519 keypair.
- She sends: `PROTOCOL_ID` ■ `Alice_ID` ■ `Alice_SID` ■ `Alice_Ephemeral_PK`

M2 (Bob → Alice):

- Bob generates a fresh 16-byte *Bob_SID* and an ephemeral X25519 keypair.
- He echoes *Alice_SID* so Alice can detect session splicing attacks.
- He sends: `PROTOCOL_ID` ■ `Bob_ID` ■ `Alice_SID` (echo) ■ `Bob_SID` ■ `Bob_Ephemeral_PK`
- Alice verifies the echoed *Alice_SID* immediately, before processing any further field.

M3 (Alice → Bob) — Authentication:

- Both sides independently build the 124-byte canonical transcript and compute SHA-256(transcript).
- Alice signs the Transcript_Hash with her long-term Ed25519 private key and sends the 64-byte signature.
- Bob verifies using Alice's trusted long-term public key. Abort rule: failure raises HandshakeError before M4 is sent.

M4 (Bob → Alice) — Authentication:

- Bob signs the same Transcript_Hash with his long-term Ed25519 private key.
- Alice verifies using Bob's trusted long-term public key. Abort rule: failure raises HandshakeError; no keys returned.

Failure handling: Any mismatch — wrong `PROTOCOL_ID`, wrong peer ID, *Alice_SID* echo mismatch, or Ed25519 verification failure — raises HandshakeError before any session key material is derived or returned to the caller.

1.3 Canonical Transcript Construction

After M2, both sides independently construct an identical 124-byte transcript:

```
Transcript = PROTOCOL_ID (12 bytes, ASCII "CS6530-A2-v1")
           || Alice_ID   (8 bytes)
           || Bob_ID     (8 bytes)
           || Alice_SID   (16 bytes)
           || Bob_SID     (16 bytes)
           || Alice_Ephemeral_PK (32 bytes, raw X25519)
           || Bob_Ephemeral_PK  (32 bytes, raw X25519)
           = 124 bytes total

Transcript_Hash = SHA-256(Transcript) → 32 bytes
```

The transcript binds both ephemeral public keys into the signatures. Any substitution of an ephemeral key (e.g., by Mallory) changes the transcript, changes the hash, and invalidates the peer's signature over that hash.

1.4 Key Derivation: X25519 + HKDF-SHA-256

After M4, both sides perform the Diffie-Hellman exchange and key derivation:

```
Step 1 – X25519 shared secret:
  shared_secret = X25519(own_eph_priv, peer_eph_pub) → 32 bytes

Step 2 – HKDF Extract (using Transcript_Hash as salt):
  PRK = HKDF-Extract(salt = Transcript_Hash, IKM = shared_secret)

Step 3 – HKDF Expand (two independent traffic keys):
  K_Alice_to_Bob = HKDF-Expand(PRK, info = "CS6530-A2 Alice->Bob", L = 32)
  K_Bob_to_Alice = HKDF-Expand(PRK, info = "CS6530-A2 Bob->Alice", L = 32)
```

- The raw X25519 shared secret is never used directly as an AES key (FR-4).
- Using Transcript_Hash as the HKDF salt binds derived keys to this session's specific IDs, SIDs, and ephemeral keys.
- The two distinct info strings produce cryptographically independent 256-bit keys, ensuring nonce independence between directions.

1.5 AES-256-GCM Record Layer: Nonce / Counter / AAD

Each direction has its own 64-bit counter starting at 0. The 96-bit GCM nonce is:

```
Nonce = 0x00000000 || uint64_be(counter) [4 + 8 = 12 bytes]
```

The Additional Authenticated Data (AAD) is:

```
AAD = Sender_ID || Receiver_ID || Alice_SID || Bob_SID || uint64_be(counter)
```

The AAD is not encrypted but is authenticated by the GCM tag. Any modification to session identifiers, message direction, or the counter byte in transit causes tag verification to fail.

Verify-then-advance ordering: the receiver checks the counter value before decryption. If AEAD decryption fails, the receive counter is NOT advanced — ensuring a garbage injection cannot consume a counter slot and block a legitimate record.

1.6 Connectivity Test

Before implementing the cryptographic protocol, the instructor-supplied test programs were verified on localhost.

```
Alice output:
  Alice: connected to Bob
  Alice received: Hello Alice

Bob output:
  Bob: connected to ('127.0.0.1', <port>)
  Bob received: Hello Bob
```

Outcome: PASS. The TCP helper, 2-byte length framing, and send_message / receive_message all function correctly prior to any cryptographic layer.

2. Testing Results

2.1 TR-1: Normal Authenticated Session

Objective:

Complete the full M1–M4 handshake, derive both directional traffic keys, and exchange at least three protected application records in each direction. Confirm that the raw X25519 shared secret is not directly used as the AES-GCM key, and that counters 0, 1, and 2 are used correctly.

Procedure:

1. Generate long-term Ed25519 keypairs for Alice and Bob; exchange public keys.
2. Alice generates Alice_SID and ephemeral X25519 keypair; sends M1.
3. Bob generates Bob_SID and ephemeral X25519 keypair; echoes Alice_SID in M2.
4. Both sides compute the 124-byte transcript and SHA-256 hash.
5. Alice signs and sends M3; Bob verifies. Bob signs and sends M4; Alice verifies.
6. Both sides compute X25519 shared secret and derive K_Alice_to_Bob, K_Bob_to_Alice.
7. Alice sends three AES-256-GCM records (counters 0, 1, 2); Bob sends three replies.
8. Verify shared_secret_fp $\neq$ k_a2b_fp (confirming HKDF was applied), and that both sides report identical fingerprints.

Test Input:

```
Alice ID: AL000001      Bob ID: B0000001
Alice_SID: e0a713436b1863e9ec39d85c08906d2c
Bob_SID:   c7e90a3e230e5dd776b225693c0c4767
Plaintext (Alice→Bob): "Alice→Bob: message 0/1/2"
Plaintext (Bob→Alice): "Bob→Alice: reply 0/1/2"
```

Expected Behaviour:

- Both sides report identical shared_secret_fp, k_a2b_fp, and k_b2a_fp.
- shared_secret_fp $\neq$ k_a2b_fp — confirming HKDF was applied and raw DH output is not the AES key.
- All six records (3 per direction) accepted. GCM tag fingerprints match across alice.jsonl and bob.jsonl.
- Nonces are 000000000000000000000000, ...01, ...02 in each direction independently.

Observed Behaviour:

Handshake completed successfully. Both alice.jsonl and bob.jsonl reported identical fingerprints: shared_secret_fp = 20fb19ff2b7f9e5f, k_a2b_fp = 6a1099fafdc55c9d, k_b2a_fp = b428f7dc68d8ceb2. shared_secret_fp (20fb19ff) differs from k_a2b_fp (6a1099fa), confirming HKDF was applied. Cross-verified: tag_fp for Alice→Bob counter-0 (28e4de03fb54eede) matches identically in both logs. All six records accepted with correct nonces and increasing counters in each direction.

Outcome: PASS

Supporting Evidence — alice.jsonl:

```
{
  "role": "alice",
  "event": "HANDSHAKE_OK",
  "alice_id": "AL000001",
  "bob_id": "B0000001",
  "alice_sid": "e0a713436b1863e9ec39d85c08906d2c",
  "bob_sid": "c7e90a3e230e5dd776b225693c0c4767",
  "shared_secret_fp": "20fb19ff2b7f9e5f",
  "k_a2b_fp": "6a1099fafdc55c9d",
  "k_b2a_fp": "b428f7dc68d8ceb2"
}

{
  "role": "alice",
  "event": "SUCCESS",
  "outcome": "RECORD_SENT",
  "direction": "AL000001→B0000001",
  "counter": 0,
  "nonce": "0000000000000000000000000000",
  "plaintext": "Alice→Bob: message 0",
  "tag_fp": "28e4de03fb54eede"
}

{
  "role": "alice",
  "event": "SUCCESS",
  "outcome": "RECORD_RECEIVED",
  "direction": "B0000001→AL000001",
  "counter": 0,
  "nonce": "0000000000000000000000000000",
  "plaintext": "Bob→Alice: reply 0",
  "tag_fp": "d39b81632c9cb704"
}

{
  "role": "alice",
  "event": "SUCCESS",
  "outcome": "RECORD_SENT",
  "direction": "AL000001→B0000001",
  "counter": 1,
  "nonce": "0000000000000000000000000001",
  "plaintext": "Alice→Bob: message 1"
}

{
  "role": "alice",
  "event": "SUCCESS",
  "outcome": "RECORD_RECEIVED",
  "direction": "B0000001→AL000001",
  "counter": 1,
  "nonce": "0000000000000000000000000001",
  "plaintext": "Bob→Alice: reply 1",
  "tag_fp": "a401395192a22737"
}

{
  "role": "alice",
  "event": "SUCCESS",
  "outcome": "RECORD_SENT",
  "direction": "AL000001→B0000001",
  "counter": 2,
  "nonce": "0000000000000000000000000002",
  "plaintext": "Alice→Bob: message 2",
  "tag_fp": "d71f6b7a61df9231"
}

{
  "role": "alice",
  "event": "SUCCESS",
  "outcome": "RECORD_RECEIVED",
  "direction": "B0000001→AL000001",
  "counter": 2,
  "nonce": "0000000000000000000000000002",
  "plaintext": "Bob→Alice: reply 2",
  "tag_fp": "d856b82d31db35af"
}
```

Supporting Evidence — bob.jsonl (HANDSHAKE_OK):

```
{
  "role": "bob",
  "event": "HANDSHAKE_OK",
  "alice_id": "AL000001",
  "bob_id": "B0000001",
  "alice_sid": "e0a713436b1863e9ec39d85c08906d2c",
  "bob_sid": "c7e90a3e230e5dd776b225693c0c4767",
  "shared_secret_fp": "20fb19ff2b7f9e5f",
  "k_a2b_fp": "6a1099fafdc55c9d",
  "k_b2a_fp": "b428f7dc68d8ceb2"
}
```

2.2 TR-2: Man-in-the-Middle (MITM) Demonstration

Objective:

First, run the protocol with Ed25519 authentication intentionally disabled (--insecure-demo). Show that Mallory can substitute ephemeral keys, establish independent sessions with Alice and Bob, and read/modify application records. Then restore full authentication and show that the same substitution is detected and the handshake aborted before any application data is exchanged.

Part A: Weakened Mode (Attack Succeeds)

Procedure:

1. Start Bob with --insecure-demo, listening on port 6540.
2. Start Mallory with --insecure-demo; she substitutes her own X25519 ephemeral keys for both sides.
3. Start Alice with --insecure-demo, connecting to Mallory's port 6541 (not directly to Bob).
4. Observe that Alice and Bob complete handshakes with Mallory, not each other.
5. Verify MALLORY_READ events show plaintext_seen for every record; Alice's k_a2b_fp ≠ Bob's k_a2b_fp.

Test Input:

```
Alice ID: AL000001    Bob ID: B0000001
Mallory listens on port 6541; connects to Bob on port 6540.
Alice connects to 127.0.0.1:6541 (Mallory), not directly to Bob.
```

Expected Behaviour:

- Both sides complete handshake without error (no authentication check performed).
- Mallory holds four independent keys: K(Alice↔Mallory) and K(Mallory↔Bob).
- Alice's k_a2b_fp matches Mallory's k_alice_to_mallory_fp — not Bob's traffic key.
- Mallory reads all six plaintext messages and re-encrypts each with "[intercepted by Mallory]" appended.

Observed Behaviour:

The MITM session completed without Alice or Bob detecting the attack. Mallory read the plaintext of all six records. Alice's k_a2b_fp (8e3cec2e) matched Mallory's k_alice_to_mallory_fp (8e3cec2e) — not Bob's k_a2b_fp (eb3d54fd) — proving Alice and Bob held different session keys. Bob received "Alice→Bob: message 0 [intercepted by Mallory]", modified by Mallory with no detection.

Outcome: PASS

Supporting Evidence — mallory.jsonl (weakened):

```
{
  "role": "mallory",
  "event": "TRANSCRIPT_HASHES",
  "alice_side_t_hash": "3da7f58e...e9fa1",
  "bob_side_t_hash": "79b49c4b...aa54c",
  "hashes_differ": true
}

{
  "role": "mallory",
  "event": "MALLORY_KEYS",
  "k_alice_to_mallory_fp": "8e3cec2e1031fee8",
  "k_mallory_to_alice_fp": "a74137b96e8cb60a",
  "k_mallory_to_bob_fp": "eb3d54fd7bed3cbe",
  "k_bob_to_mallory_fp": "210b195a44e29887"
}

{
  "role": "mallory",
  "event": "MALLORY_READ",
  "direction": "AL000001→mallory→B0000001",
  "counter": 0,
  "plaintext_seen": "Alice→Bob: message 0"
}

{
  "role": "mallory",
  "event": "MALLORY_READ",
  "direction": "B0000001→mallory→AL000001",
  "counter": 0,
  "plaintext_seen": "Bob→Alice: reply 0"
}

{
  "role": "mallory",
  "event": "MALLORY_READ",
  ... (counters 1 and 2 similarly)
}

{
  "role": "mallory",
  "event": "SUCCESS",
  "outcome": "MITM_SESSION_COMPLETE",
  "records_intercepted": 3
}
```

Part B: Authenticated Mode (Attack Fails)

Procedure:

1. Repeat the same three-process setup without --insecure-demo.
2. Mallory still substitutes ephemeral keys as before.
3. Alice's Transcript_Hash covers Mallory's A-side key; Bob's covers Mallory's B-side key — they differ.
4. Alice signs her Transcript_Hash and sends M3; Mallory cannot forward it to Bob.
5. Bob would try to verify Alice's signature against a different hash — verification would fail.
6. Mallory detects the mismatch and aborts before forwarding M3.

Test Input:

Same three-process setup. No --insecure-demo flag.

Expected Behaviour:

- Mallory logs hashes_differ: true for the two transcript hashes.
- Mallory logs FAILED with reason 'transcript mismatch' and aborts before forwarding M3.
- Alice and Bob both receive connection closed and log FAILED.
- Zero application records are exchanged.

Observed Behaviour:

Mallory immediately computed diverging transcript hashes after both sub-handshakes reached M2. Mallory logged two FAILED events and closed both connections. Alice and Bob both logged HandshakeError: 'Connection closed before complete message was received.' No application records were exchanged in either direction.

Outcome: PASS**Supporting Evidence — mallory.jsonl (authenticated):**

```
{ "role": "mallory", "event": "TRANSCRIPT_HASHES",  
  "alice_side_t_hash": "5f62dbb4ea4bfcdbd7ff8d33beb26c13b5cb62a69c8f488ccac717b1cb78d4b06",  
  "bob_side_t_hash": "7fd7885c850df53c79180e82f386a60a8e26736dc35a71a4cc6567ba6addf887",  
  "hashes_differ": true }  
  
{ "role": "mallory", "event": "FAILED",  
  "reason": "authenticated mode: M3 cannot verify at Bob (transcript mismatch)",  
  "alice_t_hash": "5f62dbb4...",  
  "bob_t_hash": "7fd7885c..." }  
  
{ "role": "mallory", "event": "FAILED",  
  "reason": "authenticated mode: aborting before forwarding M3 (cannot produce valid sig)" }
```

2.3 TR-3: Replay Attack

Objective:

Capture a valid AES-256-GCM application record (counter 0) and attempt to replay it after the receiver has advanced to counter 2. Confirm the replay is rejected because the counter is stale. Additionally demonstrate that a failed AEAD verification does not advance the receive counter (authenticate-before-advance property).

Experiment A: Stale Counter Rejection

Procedure:

1. Run a full authenticated session; capture the byte-exact wire frame for counter-0 (tag is valid and unmodified — no tampering).
2. Send counter-0 and counter-1 records normally; both are accepted. Receiver's next expected counter is 2.
3. Replay the captured counter-0 wire frame to the receiver unchanged.
4. Observe rejection at the counter check — before AEAD decryption is attempted.
5. Confirm receive_counter remains at 2 (no state consumed by the replay attempt).

Test Input:

```
Captured wire frame: counter = 0, valid GCM tag, plaintext = "record 0"  
Receiver state at time of replay: receive_counter = 2
```

Expected Behaviour:

- Counter-0 and counter-1 are accepted normally.
- Replayed counter-0 rejected: 'stale/unexpected counter: got 0, expected 2'.
- receive_counter remains 2 after rejection — no state consumed.
- Note: the GCM tag of the replayed record is still cryptographically valid. The counter check, not AEAD, causes the rejection.

Observed Behaviour:

Counter-0 and counter-1 were accepted with matching tag fingerprints. The replayed counter-0 was rejected immediately at the counter check with 'stale/unexpected counter: got 0, expected 2'. REPLAY_COUNTER_STATE confirmed receive_counter = 2, unchanged = true.

Outcome: PASS

Experiment B: Authenticate-Before-Advance

Procedure:

1. Send a garbage ciphertext (all bytes XOR 0xFF — completely invalid) at counter 0 to a receiver expecting counter 0.
2. Observe AEAD failure; confirm receive_counter is still 0 after rejection.

3. Immediately send the legitimate counter-0 record.
4. Confirm it is accepted and receive_counter advances to 1.

Expected Behaviour:

- Garbage at counter-0 → AEAD verification failed; receive_counter_after = 0 (unchanged).
- Legitimate counter-0 immediately after → accepted; receive_counter_after = 1.

Observed Behaviour:

The garbage packet caused AEAD decryption to fail; receive_counter was not advanced. The legitimate counter-0 record was then accepted successfully. This confirms that a failed AEAD attempt cannot consume a counter slot and block a legitimate record.

Outcome: PASS**Supporting Evidence — replay.jsonl:**

```
{
  "role": "replay",
  "event": "SESSION",
  "alice_sid": "6ba03e868d97acd7e7a53863f401f55d",
  "bob_sid": "647746197cc69a5dba6eb18d47f3a8fb",
  "k_a2b_fp": "7d5d679354678cbf",
  "k_b2a_fp": "8a65b7092e0b8fdf"
}

{
  "role": "replay",
  "event": "SUCCESS",
  "outcome": "RECORD_RECEIVED",
  "direction": "AL000001→B0000001",
  "counter": 0,
  "nonce": "0000000000000000000000000000",
  "plaintext": "record 0",
  "tag_fp": "efb26d326072547b"
}

{
  "role": "replay",
  "event": "SUCCESS",
  "outcome": "RECORD_RECEIVED",
  "direction": "AL000001→B0000001",
  "counter": 1,
  "nonce": "0000000000000000000000000001",
  "plaintext": "record 1",
  "tag_fp": "43621a2aac593b1b"
}

{
  "role": "replay",
  "event": "REJECTED",
  "direction": "AL000001→B0000001 (REPLAY)",
  "counter": 0,
  "reason": "stale/unexpected counter: got 0, expected 2"
}

{
  "role": "replay",
  "event": "REPLAY_COUNTER_STATE",
  "receive_counter": 2,
  "unchanged": true
}

{
  "role": "replay",
  "event": "REJECTED",
  "outcome": "GARBAGE_AT_COUNTER_0",
  "counter": 0,
  "reason": "AEAD verification failed",
  "receive_counter_after": 0
}

{
  "role": "replay",
  "event": "SUCCESS",
  "outcome": "LEGITIMATE_RECORD_ACCEPTED_AFTER_GARBAGE",
  "counter": 0,
  "plaintext": "legitimate record 0",
  "receive_counter_after": 1
}
```

2.4 TR-4: Forward Secrecy

Objective:

Run session S1 normally and capture one application record ciphertext. Simulate a later compromise of Alice's long-term Ed25519 private key and demonstrate that the S1 traffic key cannot be reconstructed from it. For the controlled positive comparison, retain Alice's ephemeral X25519 private key and show it does permit reconstruction of the S1 traffic key, proving the session key derivation chain anchors exclusively to the ephemeral key.

Procedure:

1. Run a full authenticated session S1 with `retain_ephemeral=True` (test-only flag).
2. Seal one S1 application record and capture its wire bytes.
3. Log S1 session details: `alice_sid`, `bob_sid`, `shared_secret_fp`, `k_a2b_fp`.
4. Negative control: confirm that Alice's long-term `Ed25519PrivateKey` has no `exchange()` method (type-system enforcement); set `reconstruction_possible = False`.
5. Positive control: use the retained `alice_eph_priv` to compute `X25519(alice_eph_priv, bob_epk)`, re-derive PRK and `K_Alice_to_Bob` via HKDF, and decrypt the captured S1 ciphertext.
6. Confirm `k_a2b_original_fp == k_a2b_reconstructed_fp` and plaintext is recovered.

Test Input:

```
S1 session:
  alice_sid: 6b49d31cf0955fdead22c1cb73a0de08
  bob_sid: 92d25bc22584265299d4f142a576fd33
  k_a2b_fp: 98ef3e3952227d99 (original, derived during S1)
Captured record: counter = 0, tag_fp = 411e472ec17fd72a
Long-term key type: Ed25519PrivateKey
Retained ephemeral: alice_eph_priv (X25519PrivateKey, in-memory only)
```

Expected Behaviour:

- Negative control: `Ed25519PrivateKey` has no `exchange()` method (`has_x25519_exchange_method = false`). `reconstruction_possible = false`.
- Positive control: `X25519(retained_eph_priv, bob_epk)` reproduces `shared_secret`.
- HKDF re-derives `k_a2b` with fingerprint identical to the original.
- Captured S1 ciphertext decrypts successfully to "S1 confidential record".

Observed Behaviour:

Negative control confirmed: `Ed25519PrivateKey` has no `exchange()` method (`has_x25519_exchange_method = false`); `reconstruction_possible = false`. Positive control succeeded: `k_a2b_reconstructed_fp = 98ef3e3952227d99`, exactly matching `k_a2b_original_fp`. The captured ciphertext decrypted successfully to `plaintext_recovered = "S1 confidential record"`. The ephemeral key was stored only in a `tempfile.TemporaryDirectory()` outside the repository; it is deleted automatically when the with-block exits. No private key bytes appear in any log file.

Outcome: PASS

Supporting Evidence — forward_secretcy.jsonl:

```
{
  "role": "forward_secretcy",
  "event": "S1_SESSION",
  "alice_sid": "6b49d31cf0955fdead22c1cb73a0de08",
  "bob_sid": "92d25bc22584265299d4f142a576fd33",
  "shared_secret_fp": "d0e2d1ea98ae0f93",
  "k_a2b_fp": "98ef3e3952227d99",
  "k_b2a_fp": "4acfb90b197329bb",
  "ephemeral_key_retained": "temporary test-only material (not logged)"
}

{
  "role": "forward_secretcy",
  "event": "S1_CIPHERTEXT_CAPTURED",
  "counter": 0,
  "tag_fp": "411e472ec17fd72a"
}

{
  "role": "forward_secretcy",
  "event": "LT_KEY_COMPROMISE_SIMULATION",
  "lt_key_type": "Ed25519PrivateKey",
  "has_x25519_exchange_method": false,
  "reconstruction_possible": false,
  "reason": "Ed25519PrivateKey is not an X25519PrivateKey. compute_shared_secret() and derive_traffic_keys() never received it as input."
}

{
  "role": "forward_secretcy",
  "event": "SUCCESS",
  "outcome": "EPHEMERAL_KEY_RECONSTRUCTS_S1",
  "k_a2b_original_fp": "98ef3e3952227d99",
  "k_a2b_reconstructed_fp": "98ef3e3952227d99",
  "keys_match": true,
  "plaintext_recovered": "S1 confidential record"
}
```

3. Test Results Summary

Test	Condition	Result	Outcome
Connectivity	Localhost TCP hello	Pass/Pass	PASS
TR-1: Authenticated session	Full M1-M4 + 3 records each direction	Both sides: identical k_{a2b_fp} , k_{b2a_fp} ; $shared_secret_fp \neq k_{a2b_fp}$	PASS
TR-2A: MITM weakened mode	Ed25519 disabled (--insecure-demo)	Mallory reads & modifies all 6 records; Alice and Bob have different key fps	PASS
TR-2B: MITM authenticated mode	Ed25519 enabled (normal)	Mallory aborts at M3; zero records exchanged	PASS
TR-3A: Stale counter	Counter-0 replayed after counter-2	REJECTED: stale counter; receive_counter unchanged	PASS
TR-3B: Garbage at expected counter	AEAD fails; slot not consumed; then legitimate accepted	REJECTED then SUCCESS	PASS
TR-4 negative: LT key compromise	Ed25519PrivateKey has no exchange()	reconstruction_possible = false	PASS
TR-4 positive: Ephemeral key	X25519 re-runs HKDF	keys_match = true; plaintext recovered	PASS
Unit tests	All modules	62/62 passed	PASS

4. Discussion

4.1 Why Authentication Prevents MITM

The Ed25519 signatures in M3 and M4 are computed over SHA-256 of the full 124-byte transcript, which includes both ephemeral public keys. A MITM who substitutes an ephemeral key also changes the transcript, which changes the hash, which makes Alice's signature invalid for Bob's transcript. Mallory cannot fix this: producing a valid signature for Bob's transcript requires Alice's Ed25519 private key, which Mallory does not hold.

This is why the signatures must cover the ephemeral keys specifically — a protocol that authenticated identities alone, while leaving ephemeral keys unauthenticated, would still be vulnerable to key substitution. TR-2 demonstrates both failure modes: without signatures the attack succeeds; with signatures, transcript divergence is detected and the session aborted before any application data is exchanged.

4.2 Why AES-GCM Alone Does Not Prevent Replay

AES-256-GCM guarantees authenticity — a valid tag proves the record was produced by the key holder and has not been modified since. It does not guarantee freshness. GCM has no record of which ciphertexts it has previously accepted, so a replayed counter-0 ciphertext has a valid tag even the second time it is submitted. The per-direction monotonic counter is the sole mechanism providing freshness.

The counter check is performed before AEAD decryption (counter-then-decrypt ordering). A failed AEAD attempt does not advance the receive counter, so an injected garbage packet cannot consume a counter slot and block a legitimate record.

4.3 Why Forward Secrecy Requires Ephemeral Keys

The session key derivation chain is:

```
alice_eph_priv → X25519(alice_eph_priv, bob_epk) → shared_secret
                → HKDF(salt=transcript_hash)      → K_Alice_to_Bob
```

Alice's long-term Ed25519 key is used exclusively in `sign(alice_lt_sk, t_hash)` during M3. It is never an input to `compute_shared_secret()` or `derive_traffic_keys()`. Once `alice_eph_priv` goes out of scope at the end of `alice_handshake()` in normal operation, the `shared_secret` cannot be recomputed by anyone. A later compromise of the long-term signing key can enable future impersonation but cannot decrypt past sessions. TR-4 demonstrates this precisely: the long-term key fails to reconstruct S1, while the retained ephemeral key succeeds.

4.4 Why Two Directional Keys Are Needed

A single shared key for both directions would require Alice and Bob to coordinate their send counters to avoid nonce reuse under the same key. With separate `K_Alice_to_Bob` and `K_Bob_to_Alice`, nonce uniqueness is a structural property: counter N under `K_Alice_to_Bob` and counter N under `K_Bob_to_Alice` are encrypted under different keys and can never collide, even if both start at 0 simultaneously.

5. Conclusion

This assignment implemented and verified a complete authenticated ephemeral key establishment protocol. The normal session test (TR-1) confirmed that both parties derive identical session keys and communicate successfully over AES-256-GCM with correct counter-based nonces and nonce values. The MITM test (TR-2) showed that disabling Ed25519 authentication allows a full active attack — Mallory read and modified all six records — while restoring authentication causes an immediate handshake abort through transcript mismatch, with zero application data exposed. The replay test (TR-3) demonstrated counter-based rejection and the authenticate-before-advance property. The forward secrecy test (TR-4) showed that only the ephemeral X25519 private key can reconstruct past session keys — the long-term Ed25519 key provides no such path. All 62 unit tests pass. The protocol meets all functional requirements FR-1 through FR-7 as specified in the assignment documentation.

References

RFC 7748 — Elliptic Curves for Security (X25519)

RFC 8032 — Edwards-Curve Digital Signature Algorithm (Ed25519)

RFC 5869 — HMAC-based Extract-and-Expand Key Derivation Function (HKDF)

NIST SP 800-38D — Recommendation for Block Cipher Modes of Operation: GCM

Python cryptography library — <https://cryptography.io>